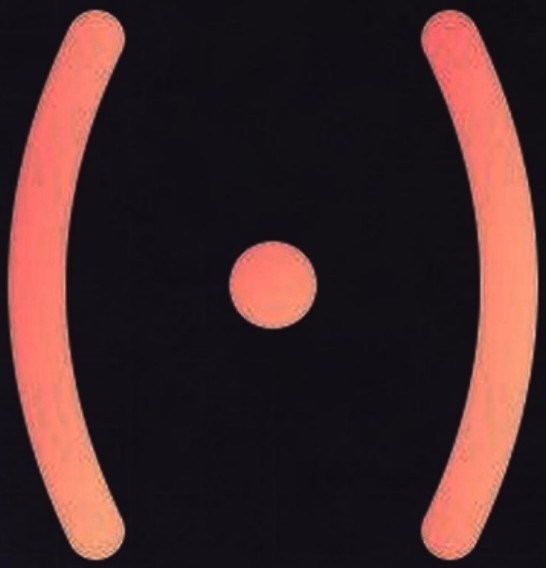
Snuggle

Security Review



1 April, 2026

Prepared by: Valves Security

Lead Auditors: Vesko210 | Merulez99

Contents

1 About Valves Security

Valves Security is a Web3 security team founded by Veselin and Valeri. With deep experience across EVM-based protocols, Cosmos (Go) chains, and native Rust systems, we help projects identify and eliminate critical vulnerabilities. With a strong background and a strong team of freelancers we can asure a top quality audit.

Book a Security Review at valvessecurity.com or message us on X Valves Security

2 Disclaimer

The Valves team makes all effort to find as many vulnerabilities in the code in the given time period. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation. We stand by our process, yet encourage follow-up audits and meaningful bug bounty incentives for continued whitehat protection.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	**Critical**	**High**	**Medium**
Likelihood: Medium	**High**	**Medium**	**Low**
Likelihood: Low	**Medium**	**Low**	**Low**

Impact

- High - leads to a significant material loss of assets in the protocol or a group of users
- Medium - leads to a moderate material loss of assets in the protocol, moderately harms a group of users or to some disruption of the protocol's functionality
- Low - No funds at risk

Likelihood

- High - attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- Medium - only a conditionally incentivized attack vector, but still relatively likely

- Low - has unlikely assumptions or requires a significant stake by the attacker

4 Protocol Summary

Snuggle is an automated yield platform that helps you earn trading fees from your crypto. Deposit Bitcoin, ETH, or stablecoins, and Snuggle's smart rebalancing technology optimizes your position to maximize fee earnings while minimizing losses.

5 Executive Summary

Overview

Project	Snuggle
Commit Hash	-
Timeline	1/04/2026 – 11/04/2026

Scope

Id	Files in scope
1	src/*

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	4
Low/Info Risk	10

6 Findings

Medium Findings

[M-01] Cross-DEX TokenId collision overwrites position in base variants

Impact: High

Likelihood: Low

Description

The vault's internal `positions` mapping is keyed by raw NFT `tokenId` with no namespace for the originating position manager contract. The Base-chain variants support multiple DEX adapters simultaneously (Uniswap V3, PancakeSwap V3, Aerodrome Slipstream), each of which maintains its own independent `NonfungiblePositionManager` contract with an independently incrementing tokenId counter starting from 1. When two different DEX pools each mint their N-th position, they produce identical tokenIds. The `_createPosition` function writes to `positions[tokenId]` unconditionally, with no guard to detect that a record already exists for that ID from a different manager:

```
// snuggle-base/SnuggleVaultUpgradeable.sol:L1077
positions[tokenId] = UserPosition({
    tokenId: tokenId,
    owner: msg.sender,
    poolId: poolId,
    // ...
});
```

When User2 deposits into a DEX pool that produces the same `tokenId` as an existing position owned by User1, `positions[tokenId]` is silently overwritten with User2's data. User1's position record — including owner address, pool ID, and tick range — is permanently destroyed without any revert or event. Secondary effects include duplicate entries in `allPositionIds` and `userPositions[user2]`, corrupting the O(1) removal invariant that relies on position indices being unique.

- User1's `withdraw(tokenId)` permanently reverts with `NotPositionOwner` because the overwritten record now points to User2 as owner.
- User1 loses total access to their deposited LP liquidity with no recovery path — the admin's `rescueERC721` cannot help because the NFT is still in the vault under the correct DEX manager, but the position record no longer recognizes User1 as owner.
- `allPositionIds` accumulates a duplicate entry, inflating `totalActivePositions` by 1 and potentially breaking downstream gas calculations that assume unique IDs.
- The collision is silent: no event, no revert. Neither user nor admin receives any on-chain signal.

Recommendation:

Add a one-line existence guard at the top of `_createPosition` before the unconditional write:

```
   function _createPosition(uint256 tokenId, bytes32 poolId, ...) internal {
       if (userPositions[msg.sender].length >= maxPositionsPerUser) revert
           MaxPositionsPerUserReached();
       if (allPositionIds.length >= maxTotalPositions) revert MaxTotalPositionsReached();
+      // Prevent cross-DEX tokenId collision from overwriting an existing position
+      if (positions[tokenId].owner != address(0)) revert PositionAlreadyExists();

       positions[tokenId] = UserPosition({...});
```

[M-02] Dust position spam due to missing minimum deposit can fill all 100,000 position slots

Impact: High

Likelihood: Low

Description

`SnuggleVaultUpgradeable._deposit()` does not enforce a minimum deposit size. Any non-zero wei amount passes validation and mints a position tracked in the vault's `allPositionIds` array and `userPositions` mapping.

The vault caps total positions at `maxTotalPositions = 100,000` and per-user at `maxPositionsPerUser = 500`. On Base and Arbitrum where gas is cheap, an attacker can create dust-liquidity positions across multiple addresses at minimal cost. Using 200 addresses with 500 positions each fills all 100,000 slots, blocking any future deposits. Estimated cost on Base: $100-$1,000 in gas plus negligible token amounts. This is going to block all future deposits in the protocol with no way to fix that.

Recommendation:

Introduce a `minDepositAmount` enforced in `_deposit()`. This raises the economic cost of position spam proportionally to the minimum, making the 100,000-position attack financially impractical.

[M-03] `calculateCenteredRange` mints one-sided, off-center positions on the minimum-width preset

Impact: Medium

Likelihood: High

Description

`TickMath.calculateCenteredRange` is supposed to wrap a symmetric band around the pool's current tick. On the minimum width preset it doesn't. Two small integer math bugs compound, and what users end up with is a one sided strip that isn't centered on anything the market cares about.

Here's the first bug. When the preset width equals a single `tickSpacing`, `halfWidth` quietly collapses to zero:

```
1  int24 halfWidth = (roundedWidth / 2 / tickSpacing) * tickSpacing;
2  // rangeWidthBps == tickSpacing → (tickSpacing / 2 / tickSpacing) * tickSpacing = 0
```

When `rangeWidthBps` equals one `tickSpacing`, the middle step of `halfWidth` computes `tickSpacing / 2 / tickSpacing`, which in real math is `0.5`. Solidity has no fractions though, so it floors the result to `0`, and `halfWidth` becomes zero. The fallback at line 57 then kicks in: `tickUpper = centerTick + tickSpacing`, `tickLower = centerTick`. The range comes out as `[centerTick, centerTick + tickSpacing]`, everything above the center and nothing below. That hits every fee tier the protocol supports (0.01%, 0.05%, 0.30%, 1%), because the minimum width on each tier is defined as one `tickSpacing`, which is exactly the trigger.

The second bug stacks on top. `centerTick` isn't the real current tick. `roundToNearestSpacing` snaps it via integer truncation toward zero, so for negative ticks it rounds upward and for positive ticks it rounds downward. Unless the current tick happens to already be a multiple of `tickSpacing`, the center is offset from the market by up to `tickSpacing 1`. That's fine on its own, but combined with bug one, that one sided chunk gets glued to the wrong side of an already offset center, and the entire range can end up sitting above or below where the market actually is.

While we're here, the same truncation quietly collapses distinct presets into the same position. On a 0.30% pool, both the 1.2% and 1.8% width presets compute `halfWidth = 60` and mint an identical ±60 tick range. Users pick "wider for safety" and get the same thing as "tighter for fees". Their choice is silently thrown away by the rounding.

Put together: the minimum preset on every fee tier produces a position that's one sided by construction and offset from the market by a rounding error, while the non minimum presets give users a menu of widths that secretly map onto half as many real positions.

Recommendation:

Reject widths that can't make a symmetric range, round half width to nearest instead of truncating, and fix `roundToNearestSpacing` to pick the genuinely closest multiple of `tickSpacing` rather than whichever is closer to zero:

```
int24 halfWidth = ((roundedWidth / 2 + tickSpacing / 2) / tickSpacing) * tickSpacing;
require(halfWidth > 0, "Range too narrow for tick spacing");
```

For defense in depth, enforce `minRangeWidthBps >= 2 * tickSpacing` at the admin config layer so the trigger value can never even be requested, and sweep existing affected positions by matching `tickUpper - tickLower == tickSpacing` against the vault's NFT set and running them through the keeper.

[M-04] Pool removal is permanently blockable by any user via dust positions

Impact: Low

Likelihood: High

Description

`removePool` iterates every active position in the vault and reverts if any belongs to the target pool:

```
function removePool(bytes32 poolId) external onlyVaultOwner {
    uint256 count = vault.totalActivePositions();
    for (uint256 i = 0; i < count; i++) {
        uint256 tokenId = vault.allPositionIds(i);
        (,bytes32 posPoolId,,,,,,,,,,,,,,) = vault.positions(tokenId);
        if (posPoolId == poolId) revert PoolHasActivePositions();
    }
    vault.removePool(poolId);
}
```

Any user can permanently prevent pool removal through two vectors:

Dust position griefing: A user opens a minimal liquidity position in the target pool and never withdraws. The position stays active indefinitely, and the owner has no mechanism to force-close it.

The vault owner loses the ability to deprecate pools.

Recommendation: Forgotten dust positions cannot be closed by the owner. Either allow pool removal while active positions exist, or add an admin function that can force-close positions while the vault is paused.

Low/Info Findings

[L-01] Retroactive performance fee application charges existing positions at new rate on already-accrued earnings

`SnuggleVaultUpgradeable` stores a single global `performanceFeeBps` variable that is applied to all fee and reward distributions. When a user deposits and creates a position, no per-position fee rate is recorded in the `UserPosition` struct. Instead, the current global rate is read at the time of every harvest, withdrawal, rebalance, or reward claim via `StakingManager.distributePerformanceFees()`.

When the vault owner changes `performanceFeeBps` via `AdminSatellite.setPerformanceFee()`, the new rate applies immediately to all existing positions' next fee distribution, including earnings that accrued entirely under the previous rate. A user who deposited when the fee was 15% and accrued trading fees over weeks will have those already-earned fees charged at the new rate (up to the 50% cap) when they next harvest or withdraw.

```
// StakingManager.sol:378 — reads current global rate, not deposit-time rate
uint256 feeBps = v.performanceFeeBps();
uint256 fee0 = (earned0 * feeBps) / BPS_DENOMINATOR;
uint256 fee1 = (earned1 * feeBps) / BPS_DENOMINATOR;
```

The setter in `AdminSatellite` enforces a 6-hour cooldown (`FEE_CHANGE_COOLDOWN`) and a maximum 500bps change per update (`MAX_FEE_CHANGE_BPS`), but does not settle existing positions first:

```
// SnuggleVaultAdminSatellite.sol:228-240
function setPerformanceFee(uint256 newFeeBps) external onlyVaultOwner {
    if (newFeeBps > MAX_PROTOCOL_FEE_BPS) revert FeeTooHigh();
    if (lastFeeChangeTime > 0 && block.timestamp < lastFeeChangeTime + FEE_CHANGE_COOLDOWN
        )
        revert FeeChangeCooldown();
    uint256 current = vault.performanceFeeBps();
    uint256 delta = newFeeBps > current ? newFeeBps - current : current - newFeeBps;
    if (delta > MAX_FEE_CHANGE_BPS) revert FeeChangeTooLarge();
    lastFeeChangeTime = block.timestamp;
    vault.setPerformanceFee(newFeeBps);
}
```

The `UserPosition` struct stores cumulative fee counters (`cumulativeFees0`, `cumulativeFees1`, `cumulativeRewards`) for APR tracking but no snapshot of the fee rate at deposit time. There is no mechanism for a user to lock in their fee rate or force settlement of accrued earnings before a fee change takes effect.

Record the `performanceFeeBps` at deposit time in the `UserPosition` struct and use that rate for the lifetime of the position. Alternatively, emit a fee change event with a delay (e.g. 7 days) so users can harvest their accrued earnings at the old rate before the new rate takes effect.

[L-02] Self-referral restriction bypass via alternate address allows users to capture their own referral fees

`SnuggleVaultUpgradeable._setReferrer()` prevents a user from setting themselves as their own referrer via a `ref == user` check that reverts with `CannotReferSelf()`. However, this restriction is trivially bypassed by supplying a second address controlled by the same person.

```
function _setReferrer(address user, address ref) internal {
    if (ref == address(0) || referrer[user] != address(0)) {
        return;
    }
```

```
    if (ref == user) {
        revert CannotReferSelf();
    }
    referrer[user] = ref;
    emit ReferrerSet(user, ref);
}
```

The referral fee is carved from the protocol's performance fee before treasury distribution. When a user sets an alt as referrer, performance fee distributions send the referral portion to the alt address instead of the treasury, **reducing protocol revenue** proportional to the number of users who exploit the bypass.

Document the known limitation or implement an allowed list of referrers.

[L-03] `approve(spender, 0)` reverts for BNB and non-standard tokens

Multiple locations use `approve(spender, 0)` or `forceApprove(spender, 0)` to reset token allowances, which reverts for BNB and other non-standard tokens whose `approve()` implementation does not return a `bool` or reverts on zero values.

1. Post-mint allowance reset in position adapters. After minting a liquidity position, all three position adapters attempt to reset residual token allowances to zero as a least-privilege measure:

```
// V15-L-02: Reset residual allowances to follow least-privilege principle
// NOTE This always revert for BNB token
if (amount0Desired > 0) IERC20(token0).forceApprove(address(nftPositionManager), 0);
if (amount1Desired > 0) IERC20(token1).forceApprove(address(nftPositionManager), 0);
```

OpenZeppelin's `forceApprove(spender, 0)` degenerates into calling the same failing operation repeatedly when the value is 0, ultimately reverting with `SafeERC20FailedOperation`. Any pool containing BNB as token0 or token1 cannot have positions minted.

Additionally, the condition `amount0Desired > 0` does not check whether a residual allowance actually exists. When the position manager consumes exactly `amount0Desired` tokens, the allowance is already 0 and the reset is unnecessary.

2. Emergency approval revocation in the vault. `emergencyRevokeApproval` uses a raw `IERC20.approve(spender, 0)` call:

```
// SnuggleVaultUpgradeable.sol:992-994
function emergencyRevokeApproval(address token, address spender) external
    onlyAdminSatellite {
    IERC20(token).approve(spender, 0);
    emit ApprovalRevoked(token, spender);
}
```

Check out this same issue reference: https://cantina.xyz/code/708eecf5-a6a0-46c1-a949-277f7408decc/findings/319

Wrap each zero-allowance reset in a try-catch and fall back to approving 1 wei when the zero-approve reverts. For example:

```
try IERC20(token0).approve(address(nftPositionManager), 0) {} catch {
    IERC20(token0).approve(address(nftPositionManager), 1);
}
```

Apply the same pattern to `emergencyRevokeApproval` in the vault. Alternatively, only attempt the reset when the residual allowance is actually non-zero.

[L-04] renounceOwnership not blocked in SnuggleVaultUpgradeable

`SnuggleVaultUpgradeable` inherits from `Ownable2StepUpgradeable` without overriding `renounceOwnership()`. Calling `renounceOwnership()` sets the owner to `address(0)` permanently, disabling every `onlyOwner` function in the vault: `pause()`, `unpause()`, `setAdminSatellite()`, and proxy upgrade authorization. This is an irreversible action with no recovery path.

Implement the same function that `KeepersHelper.sol` has

```
// KeepersHelper.sol:L84-85
function renounceOwnership() public pure override {
    revert("renounceOwnership disabled");
}
```

[L-05] Fee-exempt users retroactively lose exemption on already-accrued earnings when status is revoked

The `feeExempt` mapping is a vault-global flag checked at the moment fees are distributed, not when earnings accrue. When a user holds `feeExempt = true`, their trading fees and staking rewards accumulate inside the DEX position without any performance fee deduction. However, if the admin revokes the exemption via `setFeeExempt(user, false)` before the user claims, all previously accrued earnings, which accumulated under the expectation of zero fees - are charged the full `performanceFeeBps` rate on the next harvest, withdrawal.

```
// StakingManager.sol:374-376 — exemption checked at distribution time
if (v.feeExempt(positionOwner)) {
    return (earned0, earned1);
}

// If not exempt, full fee is applied to ALL accumulated earnings
uint256 feeBps = v.performanceFeeBps();
uint256 fee0 = (earned0 * feeBps) / BPS_DENOMINATOR;
```

The `setFeeExempt` function applies the change immediately with no settlement of pending earnings:

```
// SnuggleVaultUpgradeable.sol:951-954
function setFeeExempt(address user, bool exempt) external onlyAdminSatellite {
    feeExempt[user] = exempt;
    emit FeeExemptUpdated(user, exempt);
}
```

Force-settle (harvest) all of the user's active positions before toggling `feeExempt` status. Alternatively, snapshot the exemption status per-position at deposit time and honor it for the position's lifetime. At minimum, add a timelock or advance notice period to `setFeeExempt` so users can claim accrued earnings before the revocation takes effect.

[L-06] Missing reward adapter validation in `updatePoolRewardAdapter`

`addPoolDirect` validates that a reward adapter is compatible with the target pool via `IRewardAdapter(rewardAdapter).validatePool(pool)`, catching configuration errors at registration time. However, `updatePoolRewardAdapter` skips this validation entirely.

This means the adapter may reference a non-existent gauge or an unregistered MasterChef, causing all subsequent staking operations on that pool to silently fail or revert.

```
function updatePoolRewardAdapter(bytes32 poolId, address newRewardAdapter) external
    onlyVaultOwner {
    // NOTE here there must be the same check as above to validate reward adapter can
        actually stake in this pool
    vault.updatePoolRewardAdapter(poolId, newRewardAdapter);
}
```

An incompatible reward adapter silently breaks staking for all positions in the affected pool. Users cannot earn rewards until the owner notices and corrects the adapter.

Uncomment and implement the validation check, mirroring the logic in `addPoolDirect`. The pool address needs to be retrieved from the vault's pool config:

```
function updatePoolRewardAdapter(bytes32 poolId, address newRewardAdapter) external
    onlyVaultOwner {
    if (newRewardAdapter != address(0)) {
        address pool = adapter.getPool(token0, token1, fee);
        try IRewardAdapter(newRewardAdapter).validatePool(pool) {}
        catch { revert RewardAdapterValidationFailed(); }
    }
    vault.updatePoolRewardAdapter(poolId, newRewardAdapter);
}
```

[L-07] Single-sided deposit path lacks TWAP/spot divergence check present on two-sided path

`SnuggleRebalanceLib.executeMint()` protects two-sided deposits from being placed during abnormal pool conditions by reverting when TWAP and spot diverge by more than `tickSpacing * 10`. The single-sided branch has no equivalent guard.

```
// snuggle-base/libraries/SnuggleRebalanceLib.sol:L124-L145
if (ctx.singleSided) {
    bool isToken0 = ctx.actualAmount0 > 0;
    int24 twapTick = ctx.adapter.getTWAPTick(ctx.pool, ctx.twapInterval);
    int24 spotTick = ctx.adapter.getCurrentTick(ctx.pool);
    int24 currentTick = SnuggleLogic.selectConservativeTick(twapTick, spotTick, isToken0);
    (tickLower, tickUpper) = TickMath.calculateSnuggleRange(
        currentTick, ctx.tickSpacing, ctx.rangeWidthBps, ctx.actualAmount0, ctx.
            actualAmount1
    );
    // ...
} else {
    int24 currentTick = ctx.adapter.getTWAPTick(ctx.pool, ctx.twapInterval);
    int24 spotTick = ctx.adapter.getCurrentTick(ctx.pool);
    {
        int24 div = currentTick > spotTick ? currentTick - spotTick : spotTick -
            currentTick;
        if (div > ctx.tickSpacing * 10) revert TWAPSpotDivergence();
    }
    // ...
}
```

`SnuggleLogic.selectConservativeTick()` is the intended single-sided mitigation: it picks `max(twap, spot)` for token0 deposits and `min(twap, spot)` for token1 deposits, placing the range on the safe side of the divergence so single-asset liquidity cannot be immediately swapped through at a manipulated price. This makes single-sided deposits resilient to price manipulation, but it does not cap the *magnitude* of divergence that will be tolerated.

When TWAP lags spot by more than `tickSpacing * 10` (roughly 1% on a `tickSpacing = 10` pool, easily reached during trending markets or liquidation cascades with no manipulation required), a two-sided depositor is

refused while a single-sided depositor is quietly placed in a range that may be hundreds of ticks from where the pool actually trades. No principal loss occurs, but the position sits out-of-range earning no fees until the next keeper rebalance or a manual `manualSnuggle()` call corrects it.

Apply the same divergence guard to the single-sided branch, so both deposit paths share the same abnormal-conditions behaviour:

```
if (ctx.singleSided) {
    int24 twapTick = ctx.adapter.getTWAPTick(ctx.pool, ctx.twapInterval);
    int24 spotTick = ctx.adapter.getCurrentTick(ctx.pool);
    {
        int24 div = twapTick > spotTick ? twapTick - spotTick : spotTick - twapTick;
        if (div > ctx.tickSpacing * 10) revert TWAPSpotDivergence();
    }
    bool isToken0 = ctx.actualAmount0 > 0;
    int24 currentTick = SnuggleLogic.selectConservativeTick(twapTick, spotTick, isToken0);
    // ...
}
```

Alternatively, document that single-sided deposits are expected to land out-of-range during high-divergence conditions and rely on the subsequent rebalance cycle.

[L-08] Cached `tickSpacing` becomes stale when Camelot/Algebra pool tick spacing is updated

`SnuggleVaultAdminSatellite.addPool()` reads the current tick spacing once at pool registration and stores it in `PoolConfig`. From that point on the vault always uses the stored value when calculating ranges, rounding ticks, and minting positions. There is no mechanism to refresh it.

```
// snuggle-base/SnuggleVaultAdminSatellite.sol:L128
int24 tickSpacing = adapter.getTickSpacing(pool);
// ...
vault.addPoolDirect(poolId, pool, token0, token1, fee, tickSpacing, ...);
```

This is safe for Uniswap V3, where tick spacing is fixed forever once a fee tier is created. It is not safe for Camelot V3 on Arbitrum, which runs on Algebra V1.9. Algebra V1.9 pools expose a `setTickSpacing(int24)` function on the pool contract itself, callable by the Algebra factory owner:

```
// AlgebraV1.9 AlgebraPool.sol:L971-L973
function setTickSpacing(int24 newTickSpacing) external override lock onlyFactoryOwner {
    require(newTickSpacing > 0 && newTickSpacing <= Constants.MAX_TICK_SPACING &&
        tickSpacing != newTickSpacing, 'Invalid newTickSpacing');
    tickSpacing = newTickSpacing;
```

When the Algebra factory owner changes a Camelot pool's tick spacing, Snuggle's cached value falls out of sync. The next time Snuggle tries to mint into that pool, `calculateSnuggleRange()` / `calculateCenteredRange()` produces ticks aligned to the stale value (e.g. multiples of 60) while the pool now requires alignment to a different value (e.g. multiples of 100), and the mint reverts at the Algebra pool level with a tick alignment error.

The result is a hard DoS on every interaction with that pool that requires minting:

- Fresh deposits revert
- Rebalances revert (the old position is burned inside the transaction, but the revert on the failing remint unwinds that burn, so existing positions stay intact)
- `manualSnuggle` reverts
- Keeper rebalances revert

Withdrawals are unaffected because they only call `decreaseLiquidity` and `collect`, neither of which requires tick alignment. Existing Algebra positions opened before the tick spacing change also continue to function per Algebra's own documentation — the DoS is confined strictly to *new* mints on the affected pool.

Likelihood is Low: the trigger requires action by the Camelot/Algebra factory owner, not any arbitrary caller. But once triggered, the affected pool is permanently stuck for deposits and rebalances until a Snuggle admin notices and re-registers the pool (full `removePool` + `addPool` cycle, operationally disruptive on a live pool).

Refresh the stored tick spacing on each interaction by reading it from the adapter at the start of `executeMint` and `executeRebalance`, rather than trusting the cached value in `PoolConfig`. The extra `staticcall` per operation is negligible compared to a stuck pool. Alternatively, add a dedicated admin function that lets the satellite update `PoolConfig.tickSpacing` in place, so recovery from this state is a single transaction rather than a full re-onboarding.

[L-09] Adding a pool with a new token breaks fee harvesting until the admin re-registers the helper

`FeeTransferHelper` moves fees out of the vault via `safeTransferFrom(vault, recipient, amount)`, so it needs an allowance on every token it will ever transfer. Allowances are only granted as a side effect of `addPoolDirect`:

```
// snuggle-base/SnuggleVaultUpgradeable.sol:L411-L412
IERC20(token0).approve(_positionAdapter, type(uint256).max);
IERC20(token1).approve(_positionAdapter, type(uint256).max);
```

The helper is registered once at deploy time with a single reference token pair, so it only has allowances on those two tokens. When the admin later calls `satellite.addPool` with a **real** adapter and a new token, the vault approves the real adapter but never the helper. The new pool looks healthy, users deposit, fees accrue — and then the first harvest reverts with zero allowance on the new token.

Impact. All fee paths revert for the affected pool: `distributePerformanceFees` (`StakingManager.sol:L358`), `processFeesForCompounding` (L299), and `_harvestAllFees` (L423). The referral fallback at L450-L452 calls the same broken helper and also reverts. User principal is safe (`withdraw()` uses `adapter.collect()` directly at L584), but accrued fees sit stranded in the vault until the admin intervenes. Nothing warns at registration time — the misconfiguration only surfaces on the first harvest.

Recovery exists but is undocumented. The admin can call `satellite.addPool` a second time with the helper as `positionAdapter` and the new pair; `addPoolDirect` then grants the helper max allowance on those tokens. The resulting `PoolConfig` is cosmetically junk but inert (every mint on the helper reverts `NotImplemented`). The deployment comment at `FeeTransferHelper.sol:L38-L44` does not mention this step.

Recommendation. Preferred fix — in `addPoolDirect`, also approve the helper whenever a new pair is added:

```
  IERC20(token0).approve(_positionAdapter, type(uint256).max);
  IERC20(token1).approve(_positionAdapter, type(uint256).max);
+ if (address(stakingManager) != address(0)) {
+     address helperAddr = address(stakingManager.feeTransferHelper());
+     if (helperAddr != address(0)) {
+         IERC20(token0).approve(helperAddr, type(uint256).max);
+         IERC20(token1).approve(helperAddr, type(uint256).max);
+     }
+ }
```

Alternatives: a dedicated `approveTokenForHelper(token)` admin function on the satellite, or at minimum a documented step 6 in the deployment comment warning operators that every new token pair requires a second `addPool` call with the helper as adapter.

[I-01] clearReferrer leaves stale data in ReferralTracker when reassigning to a different referrer

The vault's clearReferrer(user) only deletes referrer[user] on the vault side. The separate ReferralTracker contract keeps its own userReferralIndex[user] mapping, which is never touched and has no admin function to clear it.

If the user is later reassigned to a different referrer, the vault accepts the change. But on the next referral fee distribution, StakingManager._safeTransferReferral sends the fee to the new referrer and then calls Referral-Tracker.recordReferral(user) to register the relationship. This call reverts with AlreadyRecorded because the stale index still points to the old referrer. The revert is swallowed by the surrounding try-catch, leaving the tracker permanently inconsistent:

vault.referrer[user] = newReferrer ReferralTracker.referrals[oldReferrer] = [user] // stale ReferralTracker.referrals[newReferrer] = [] // missing Payouts and earnings accounting still work correctly because both read from the vault, not from userReferralIndex. The broken pieces are the on-chain relationship queries: getReferrals, getReferralCount, and isReferralRecorded return stale or misleading data for the affected user. Reassigning to the same original referrer is not affected.

Recommendation: Add a clearReferral(address user) function to ReferralTracker, restricted to the vault owner, and invoke it from clearReferrer so both sides are reset atomically. Alternatively, drop the AlreadyRecorded revert in recordReferral and let it re-sync whenever the vault's referrer has changed.